

Due Sunday September 27th

1. This year is an election year, and it looks like it might be a close one, so here are some election statistics to ponder.

Suppose a million votes are cast for two opposing candidates. Suppose that voters are *slightly* less likely to vote for one candidate than for the other. Specifically, assume that on average 49.9% of voters will choose the less favored candidate. What are the chances that there is a tie (each candidate gets exactly 500,000 votes)? What are the chances that the less-favored candidate will actually win?

To answer these questions, assume that the votes for the less-favored candidate follow a binomial distribution such that the chance P_B of the candidate getting exactly k votes out of a total of n votes is

$$P_B(k; n, p) = \frac{n!}{k!(n-k)!} p^k (1-p)^{(n-k)}$$

where $p = 0.499$ is the probability that any individual voter will vote for this candidate. This equation may be difficult to solve numerically, owing to the large numbers involved. To mitigate this difficulty, you may want to make use of the approximation to the binomial distribution given in Exercise 3 of Lab 5, which should be very good for p near 0.5 and $n = 1,000,000$.

Write a Python script **elect.py** which takes command-line input from the user to specify p . Have your code print out the following:

```
probability p that voter chooses candidate: XXX
probability of tie:                        YYY
probability of win:                        ZZZ
```

Note: this probabilistic interpretation of voters does not mean that voters themselves necessarily act in a random fashion. Instead, probabilities are assigned because the many influences on individual choices prevent us from knowing much more than how voters behave on average, as gleaned from surveys and polls. In this sense the situation is similar to a coin-flipping experiment, wherein unpredictable processes influence the outcome and we only know (if the coin is a fair/"honest" one) that on average, we'll get 50% heads. ($p=0.5$ in the above equation). Play around with different input values for p . I think it is amazing how finely tuned p must be for there to be a close election!

2. Suppose you needed to generate a Gaussian (normal) distribution

$$f(x|\mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-(x-\mu)^2/2\sigma^2}$$

of N_{RV} random variates, but the only random number generator you know about is `numpy.random.uniform(0,1,N)`. However, assume you do know about the central limit theorem. Create a Python script **gengauss.py** which does the following:

- Reads μ , σ and N_{RV} from the command line. Be sure that your script checks that it has the proper inputs, and prompts the user if not.
- Uses the central limit theorem to generate N_{RV} values of x having the proper distribution, using only random numbers from `numpy.random.uniform(0,1,N)`.
- Creates a plot in the file **gengauss.png**, showing a histogram of x values with the PDF $f(x|\mu, \sigma)$ superimposed.

Be sure the distribution you generate has mean and width as specified on the command line.

3. The distribution of decay times for elementary particles is well-modeled by the *exponential distribution*, with PDF given by:

$$f(x|\tau) = \frac{1}{\tau} e^{-t/\tau}$$

t is the decay time and τ is the lifetime parameter of the elementary particle.

The file **muons20051130.dat** contains data from an experiment to measure the lifetime of the elementary particle known as the muon. (A similar experiment lives in South Physics, and if you take PHYS 3719 you may have an opportunity to perform this experiment.) The file contains a single column with the decay time, in microseconds, of a set of cosmic ray muons recorded on November 30, 2005.

For this problem, you are to determine the muon lifetime, with uncertainties and goodness-of-fit using the χ^2 minimization method. You should create a Python script **muon-chi2.py** to perform the following:

- Open the data file and sort decay times into bins. I recommend using 25 bins between 0 and 8 μsec .
- Create independent variable x from the central value of the bins, and dependent variable y from the number of entries in each bin. Use the square root of the number of entries as the (assumed Gaussian) random error in the dependent variable.
- Use `scipy.optimize.curve_fit` to extract the best fit value for τ . Compute χ^2/df and goodness of fit. Print your results to standard output.

Submit your Python script **muon-chi2.py**.

4. Now, let's approach the same data as Problem 2 using the maximum likelihood method. You should create a Python script **muon-maxl.py** to perform the following:
- Open the data file and store the unbinned decay times.
 - (Note that, in your function that calculates log likelihood, you have to be a little careful to make sure that your PDF is normalized to unity.)
 - For values of the parameter τ in the range $1.7 < \tau < 2.3$ microseconds, with step sizes $\Delta\tau = 0.01$ microseconds, compute the log-likelihood. Make a plot of log-likelihood versus τ , and find the best-fit value of τ . Find the uncertainty in τ by finding the values τ_- and τ_+ at which the log-likelihood decreases by $1/2$ (an absolute number, not fractional change!) relative to the maximum. Note that your uncertainties may not be symmetric.
 - Print your results to standard output and create a final plot **muon-maxl.png** of the data overlaid with your best fit.

Submit your Python script **muon-maxl.py**.

Submit your assignment by 11:59 PM on Sunday, September 27th:

submit p4730 hw02 elect.py gengauss.py muon-chi2.py muon-maxl.py